



не для прямой речи. Ниже подготовлены три промпта под три стратегии из теории: «человек в контуре», «человек на ревью», «автономный агент с лимитами». Все запросы можно отправлять в обычный чат. Используйте один и тот же код и три разных запроса. После получения ответов сравните:

Насколько сильно меняется самостоятельность модели при одинаковой задаче.

Какая стратегия даёт больше контроля разработчику.

Какая стратегия быстрее приводит к готовому результату.

Где требуется больше проверки человеком.

Какие ошибки может допустить ИИ при каждом варианте взаимодействия.

Основная цель практики — увидеть, что качество работы с ИИ зависит не только от самой модели, но и от выбранной стратегии взаимодействия.

Исходный материал для всех стратегий

Используем одну функцию:

```
def calculate_price(items, discount):  
    result = 0  
  
    for item in items:  
        result += item["price"] * item["count"]  
  
    result = result - result * discount / 100  
  
    return result
```

Задача: улучшить код функции расчёта стоимости заказа.

Стратегия 1. «Человек в контуре»

В этой стратегии человек постоянно участвует в процессе. ИИ не получает задачу полностью выполнить самостоятельно, а выступает как помощник разработчика.

Промпт:

Я работаю над Python-функцией расчёта стоимости заказа.

Проанализируй этот код и предложи возможные улучшения:

```
def calculate_price(items, discount):
```

```
result = 0

for item in items:
    result += item["price"] * item["count"]

result = result - result * discount / 100

return result
```

Не переписывай функцию полностью.

Сначала объясни:

- какие проблемы ты видишь;
- какие улучшения можно внести;
- какие варианты решения существуют.

После анализа предложи один улучшенный вариант функции и объясни, почему он лучше исходного.

Ожидаемый формат работы: модель анализирует код и предлагает варианты. Человек самостоятельно выбирает, какие изменения принять.

Стратегия 2. «Человек на ревью»

В этой стратегии ИИ получает больше свободы. Он готовит результат, а человек проверяет итог.

Промпт:

Выполни рефакторинг этой Python-функции расчёта стоимости заказа.

Исходный код:

```
def calculate_price(items, discount):
    result = 0

    for item in items:
        result += item["price"] * item["count"]

    result = result - result * discount / 100

    return result
```

Подготовь улучшенную версию функции.

После этого предоставь краткий отчёт:

- какие изменения были сделаны;
- какие проблемы исходного варианта исправлены;
- какие возможные ограничения остались.

Сохрани первоначальное назначение функции и её ожидаемое поведение.

Ожидаемый формат работы: модель сразу создаёт вариант решения, после чего разработчик выполняет проверку результата.

Стратегия 3. «Автономный агент с лимитами»

В этой стратегии ИИ получает цель и должен самостоятельно пройти несколько этапов: анализ, планирование изменений, выполнение рефакторинга и проверку результата.

Промпт:

Требуется улучшить качество функции расчёта стоимости заказа.

Работай как разработчик, которому необходимо подготовить изменение для проекта.

Исходный код:

```
def calculate_price(items, discount):  
    result = 0  
  
    for item in items:  
        result += item["price"] * item["count"]  
  
    result = result - result * discount / 100  
  
    return result
```

Цель:

сделать функцию более надёжной, читаемой и удобной для дальнейшей поддержки.

Перед изменением:

проанализируй текущую реализацию и определи возможные проблемы.

Затем:

подготовь план изменений;
выполни рефакторинг;
проверь, что новая версия сохраняет исходное поведение.

Ограничения:

не изменяй назначение функции;

не добавляй сторонние библиотеки;
не усложняй решение без необходимости.

В конце предоставь:
результат изменений;
краткий отчёт о выполненных действиях;
какие проверки были выполнены.

Ожидаемый формат работы: модель пытается имитировать процесс агента — сначала определить шаги, затем выполнить изменения и провести самопроверку.
